

Übungen zur Textverarbeitung mit der bash – Lösungen

grep, sort, head, tail, uniq, cut

Aufgabe 1:

Testen Sie Ihre regexp zur Erkennung richtiger IP-Adressen mit dem grep-Kommando und dem File `IPAddress`.

Aufgabe 2:

Verwenden Sie für folgende Aufgaben den File `abfahrt` und geben Sie die entsprechenden Bash - Kommandos zur Lösung folgender Aufgaben an:

- 1) Wie viele Österreicher waren am Start?
- 2) Wie viele Amerikaner befinden sich unter den TOP 10?
- 3) Wie viele Fahrer haben unter 1 sec Rückstand auf den Sieger des Rennens?
- 4) Wie viele Fahrer haben mehr als 1 sec Rückstand auf den Sieger des Rennens?
- 5) Aus welchem Land kommen die meisten Teilnehmer?
- 6) Wie viele Abfahrer haben an diesem Rennen teilgenommen und sind klassifiziert worden?
- 7) Wie viele Abfahrer stammen aus Österreich oder Italien?
- 8) Wie viele Abfahrer sind disqualifiziert worden?
- 9) Auf welchen Plätzen wurden Teilnehmer zeitgleich platziert? Geben Sie nur Anzahl und Platzierung aus!

Praktische Übungen mit dem sed:

Verwenden Sie für die nachfolgenden Übungen den File `besucher`.

- 1) Löschen Sie die erste Zeile und alle Leerzeilen aus dem File `besucher`.
- 2) Ersetze die Zeichenfolge `' - - '` durch ein einzelnes Leerzeichen.
- 3) Ersetze die Zahlen am Ende jeder Zeile durch Nichts (= Lösche die Zahlen am Ende jeder Zeile)
- 4) Ersetze in jeder Zeile, die mit `ics2f` beginnt, die Zahl `06` durch die Zahl `16`.
- 5) Ersetze in den Zeile 1 bis 5 die Zahl `06` durch die Zahl `16`.
- 6) Löschen Sie alle Zeilen, die mit `modem05.pbnet.com.br` beginnen.
- 7) Löschen Sie alle Zeilen zwischen dem ersten Auftreten von NICHT KLAUSURRELEVANT `modem05.pbnet.com.br` und der Zeile 10.
- 8) Ersetzen Sie alle Datumsangaben von `06/Nov/1997` auf `6.11.1997`

1) `sed -E '1d' besucher | sed -E '/^$[[[:space:]]*/d' besucher` --- note: d steht für delete line2) `sed -E`

Shellprogrammierung

Aufgabe:

a)

Folgender File *studenten* ist gegeben.

```
1410307047 Aberl David
1410307048 Aigner Martin Engelbert
1410307104 Amadi Christoph Chukwuka
1410307050 Baumgartner David
1420307106 Berei Tobias
1410307051 Bernard Michael Pascal
1410307052 Eder Fabian Helmut
1420307107 Enzesberger Hannes
1410307053 Exler Markus Peter Dietmar
1410307055 Giritzer Daniela
1410307056 Golledge Mathew Kyle
1410307105 Haasdyk Timothy Norman
```

Erstellen Sie ein Shell-Script *email*, das eine Datei *stud_email* mit der Email-Adresse der Studierenden nach unten dargestelltem Schema erzeugt!

```
1410307047 S1410307047@students.fh-hagenberg.at Aberl David
1410307048 S1410307048@students.fh-hagenberg.at Aigner Martin Engelbert
1410307104 S1410307104@students.fh-hagenberg.at Amadi Christoph Chukwuka
1410307050 S1410307050@students.fh-hagenberg.at Baumgartner David
1420307106 S1420307106@students.fh-hagenberg.at Berei Tobias
1410307051 S1410307051@students.fh-hagenberg.at Bernard Michael Pascal
1410307052 S1410307052@students.fh-hagenberg.at Eder Fabian Helmut
1420307107 S1420307107@students.fh-hagenberg.at Enzesberger Hannes
1410307053 S1410307053@students.fh-hagenberg.at Exler Markus Peter Dietmar
1410307055 S1410307055@students.fh-hagenberg.at Giritzer Daniela
1410307056 S1410307056@students.fh-hagenberg.at Golledge Mathew Kyle
1410307105 S1410307105@students.fh-hagenberg.at Haasdyk Timothy Norman
```

b)

Erweitern Sie dieses Shellsript *email* so, dass die Mail-Adressen der Studierenden nicht nur auf Basis der Matrikelnummern erstellt werden, sondern auch nach folgendem Schema: **Vorname.Name@students.fh-hagenberg.at**

```
1410307047 S1410307047@students.fh-hagenberg.at David.Aberl@students.fh-hagenberg.at Aberl David
1410307048 S1410307048@students.fh-hagenberg.at Martin.Aigner@students.fh-hagenberg.at Aigner Martin Engelbert
1410307104 S1410307104@students.fh-hagenberg.at Christoph.Amadi@students.fh-hagenberg.at Amadi Christoph Chukwuka
1410307050 S1410307050@students.fh-hagenberg.at David.Baumgartner@students.fh-hagenberg.at Baumgartner David
1420307106 S1420307106@students.fh-hagenberg.at Tobias.Berei@students.fh-hagenberg.at Berei Tobias
1410307051 S1410307051@students.fh-hagenberg.at Michael.Bernard@students.fh-hagenberg.at Bernard Michael Pascal
1410307052 S1410307052@students.fh-hagenberg.at Fabian.Eder@students.fh-hagenberg.at Eder Fabian Helmut
1420307107 S1420307107@students.fh-hagenberg.at Hannes.Enzesberger@students.fh-hagenberg.at Enzesberger Hannes
1410307053 S1410307053@students.fh-hagenberg.at Markus.Exler@students.fh-hagenberg.at Exler Markus Peter Dietmar
1410307055 S1410307055@students.fh-hagenberg.at Daniela.Giritzer@students.fh-hagenberg.at Giritzer Daniela
1410307056 S1410307056@students.fh-hagenberg.at Mathew.Golledge@students.fh-hagenberg.at Golledge Mathew Kyle
1410307105 S1410307105@students.fh-hagenberg.at Timothy.Haasdyk@students.fh-hagenberg.at Haasdyk Timothy Norman
```

c)

Übergeben Sie den File *studenten* als Argument an das ShellScript und überprüfen Sie, ob dieser File auch existiert. Überprüfen Sie auch, ob dem Shellscript überhaupt ein Filename übergeben wurde. Beenden Sie das Shell Script mit einer Fehlermeldung, wenn es diesen File nicht gibt bzw. kein Argument übergeben wurde.

Regex Übungen

Folgender Text ist gegeben. Sie können diesen Inhalt direkt über den File `inhalte_für_regex` verwenden.

Der Studiengang Software Engineering feierte 2013 sein 20-jähriges Bestehen!

Die Jubiläumsfeier fand am Samstag, 14. September 2013 um 16:00 Uhr in Hagenberg statt, im Mittelpunkt standen unsere Absolventinnen und Absolventen.

Im Vollzeit-Studium Software Engineering entscheiden sich die Studierenden nach dem ersten Studienjahr zwischen den beiden wahlpflichtigen Vertiefungsmodulen Business Software und Web Engineering.

In der berufsbegleitenden Form des Bachelors Software Engineering ist eine Vertiefung im Bereich Web Engineering Teil des Studienplans.

Die Lehrveranstaltungen der Vollzeitform finden von Montag bis Freitag in der Zeit von 8:00 bis 17:50 statt.

Die berufsbegleitenden Lehrveranstaltungen des Studiengangs Software Engineering finden an Freitagen von 15.30 bis 19.40 Uhr bzw. 20.30 Uhr statt, an Samstagen wird von 8.00 bis 14.40 Uhr gelehrt.

Manche Lehrveranstaltungen können über E-Learning-Module abgewickelt werden.

Weitere Informationen zum Studium unter:

FH OÖ Studienbetriebs GmbH
Fakultät für Informatik/Kommunikation/Medien
Softwarepark 11, 4232 Hagenberg/Austria
Tel.: +43 (0)50804-22001 (Vollzeit)
bzw. +43 (0)50804-22201 (berufsbegleitend)
Fax: +43 (0)50804-22099
E-Mail: se@fh-hagenberg.at
Web: www.fh-ooe.at/se

Verwenden Sie für diese Übung folgenden Link: <http://www.regexr.com/>

Dort befindet sich ein Regex Simulator mit dem Sie regexps testen können. Kopieren Sie den oben angegebenen Text (Inhalt im File `inhalte_fuer_regex`) in den Text Bereich und dann formulieren Sie für folgende Fragestellungen richtige regexps:

- 1) Finden Sie alle Textstellen, die das Wort Engineering beinhalten. Engineering
- 2) Finden Sie alle Textstellen, die die Worte Software Engineering oder Web Engineering beinhalten.
- 3) Finden Sie alle kleingeschriebenen deutschen Umlaute: ä ü ö.
- 4) Finden Sie alle Großbuchstaben und ÖÜÄ im Text.
- 5) Finden Sie alle Worte, die großgeschrieben sind. Es soll das ganze Wort ausgegeben werden!

- 7) Finden Sie alle Worte, die mit einem Großbuchstaben beginnen und mit einem Großbuchstaben enden.
- 8) Finden Sie alle Worte in Klammern.
- 9) Finden Sie alle Worte, die genau aus zwei Großbuchstaben bestehen.
- 10) Finden Sie alle gültigen INTEGER Zahlen.
- 11) Finden Sie alle Telefon- bzw. Faxnummern im Text.
- 12) Finden Sie alle Textstellen, deren Worte durch / getrennt sind.
- 13) Finden Sie alle Textstellen, deren Worte durch - getrennt sind.
- 14) Finden Sie alle gültigen Zeitangaben.

Zusätzliche Übungsbeispiele:

- 1) Finden Sie alle Worte, die genau aus drei Kleinbuchstaben bestehen
- 2) Finden Sie alle Begriffe, die mit einem Doppelpunkt begrenzt sind.
- 3) Suchen Sie gendergerechten Formen von Hauptworten.
- 4) Finden Sie oberösterreichische Postleitzahlen.
- 5) Finden Sie Wochentage im Text
- 6) Finden Sie URLs, die mit www. beginnen
- 7) Suchen Sie gendergerechte Formen von Hauptworten (Studenten, Absolventen).

Übungen Shellprogrammierung

Aufgabe 1:

Erstellen Sie ein Shellsript mmv (=multiple move), das die Extensions von Filenamen, die sich in im aktuellen Arbeitsverzeichnis befinden, ändert. Überprüfen Sie auch die Anzahl der Argumente.

Beispiel:

mmv htm html --- Umbenennen aller Files mit der Extension .htm in .html

Anmerkung: Unterverzeichnisse sind nicht zu ändern!

Aufgabe 2:

Der File schi_gebiet ist gegeben und umfasst insgesamt **vier** Schigebiete. Pro Schigebiet werden der **Name** des Schigebiets, die beteiligten **Ortschaften**, die **Anzahl der Lifte** und der **Preis der Tageskarte** erfasst. Zwischen den einzelnen Schigebieten befinden sich Leerzeilen (siehe Darstellung unten):

Großglockner Resort Orte: Matrei in Osttirol,Kals am Großglockner,Großdorf,Lesach,Virgen Lifte/Bahnen: 13 Skipass: € 45 (Tageskarte Hauptsaison)
Lermoos – Grubigstein Orte: Lermoos,Lähn Lifte/Bahnen: 8 Skipass: € 43 (Tageskarte Hauptsaison)
Sölden Orte: Sölden,Hochsölden,Zwieselstein,Huben,Längenfeld Lifte/Bahnen: 33 Skipass: € 52 (Tageskarte Hauptsaison)
Ski Juwel Alpbachtal Wildschönau Orte: Inneralpbach,Auffach,Alpbach,Oberau,Reith im Alpbachtal,Brixlegg Lifte/Bahnen: 46 Skipass: € 44 (Tageskarte Hauptsaison)

Schreiben Sie ein Shell Script formatieren, dass den File schi_gebiet entsprechend der Darstellung unten umformatiert und im File schigebiet_2 speichert.

Großglockner Resort;13;€ 45 (Tageskarte Hauptsaison)
Matrei in Osttirol, Kals am Großglockner, Großdorf, Lesach, Virgen
Lermoos – Grubigstein;8;€ 43 (Tageskarte Hauptsaison)
Lermoos, Lahn
Sölden;33;€ 52 (Tageskarte Hauptsaison)
Sölden, Hochsölden, Zwieselstein, Huben, Längenfeld
Ski Juwel Alpbachtal Wildschönau;46;€ 44 (Tageskarte Hauptsaison)
Inneralpbach, Auffach, Alpbach, Oberau, Reith im Alpbachtal, Brixlegg

1. Der File schi_gebiet wurde unter Windows generiert.
2. Überprüfen Sie, ob der File schi_gebiet wirklich existiert.
3. Pro Schigebiet sind zwei Zeilen vorzusehen. In der ersten Zeile sind Name des Schigebiets, Anzahl der Lifte und Preis der Tageskarte vorzusehen. Die einzelnen Felder sind durch Strichpunkte zu trennen. In der zweiten Zeile sind die am Schigebiet beteiligten Ortschaften zu listen, die jeweils durch Beistriche voneinander zu trennen sind.
4. Die Ortschaften pro Schigebiet sind zusätzlich einzeln in den File gemeinde zu schreiben. Dieser File soll die Ortschaften aller Schigebiete beinhalten, er ist vor Ende des Shell Scripts absteigend sortiert auszugeben.

Aufgabe 1:

Der folgende File `zuege` enthält einen Auszug des Zugverkehrs von Linz nach Wien. Er enthält die Spalten Station, Datum, ab/an , Uhrzeit der Abfahrt in Linz, Ankunftszeit in Wien, die Dauer der Reise und die Zugbezeichnung unter Verkehrsmittel.

Trennzeichen zwischen den Feldern sind Tabs, innerhalb der Felder Leerzeichen.

Station	Datum	ab/an	Zeit	Dauer	Zugnummer
Linz/Donau Hbf	11.05.2015	ab	09:30	01:34	IC 547
Wien Hbf		an	11:04		
Linz/Donau Hbf	11.05.2015	ab	09:45	01:27	ICE 323
Wien Hbf		an	11:12		
Linz/Donau Hbf	11.05.2015	ab	10:02	01:20	WB 911
Wien Westbahnhof		an	11:22		
Linz/Donau Hbf	12.05.2015	ab	10:15	01:15	RJ 861
Wien Westbahnhof		an	11:30		
Linz/Donau Hbf	12.05.2015	ab	10:30	01:34	IC 549
Wien Westbahnhof		an	12:04		
Linz/Donau Hbf	12.05.2015	ab	11:02	01:20	WB 913
Wien Westbahnhof		an	12:22		
Linz/Donau Hbf	13.05.2015	ab	11:15	01:15	RJ 765
Wien Westbahnhof		an	12:30		
Linz/Donau Hbf	13.05.2015	ab	11:30	01:34	IC 693
Wien Westbahnhof		an	13:04		
Linz/Donau Hbf	13.05.2015	ab	11:45	01:27	ICE 21
Wien Hbf		an	13:12		

Welche Linux Kommandos können nachfolgende Aufgaben umsetzen:

- Wie viele Züge kommen am Wiener Westbahnhof an?
- Erstellen Sie eine Liste `abfahrt`, die all die Züge enthält, die am 12.05 oder am 13.05. von Linz abfahren.
- Wie viele Züge fahren unter 1:25 nach Wien?
- Erstellen Sie eine Liste aller Züge in einer Datei `abfahrt_vormittag`, die vor 12:00 von Linz abfahren!
- Erstellen Sie eine numerisch sortierte Liste `verkehrsmittel`, die nur die Verkehrsmittel für Züge enthält, die als RJ (=Railjet) oder von der WB (=Westbahn) geführt werden. Die Liste `verkehrsmittel` soll folgendermaßen aussehen:

```
RJ 765
RJ 861
WB 911
WB 913
```

- f) Löschen Sie die erste Zeile des Files!
- g) Ändern Sie die Datumsangabe von xx.05.2015 auf xx.Mai 2016
- h) Löschen Sie alle Zeilen, die Ankunftszeiten enthalten.
- i) Ersetzen Sie Linz/Donau Hbf durch Linz Hbf!

Aufgabe 2:

Autokennzeichen:

Ein reguläres, österreichisches KFZ-Kennzeichen der aktuellen **EU Serie V, einzeilig** darf maximal insgesamt sieben Stellen aufweisen. Das Landeswappen teilt das KFZ-Kennzeichen in zwei Bereiche und wird für die Berechnung der Gesamtlänge nicht mitgezählt.

Links vom Wappen kommt die Abkürzung der Zulassungsbehörde, die aus ein oder zwei Großbuchstaben besteht. Dann folgt das Landeswappen.

Für den Bereich rechts vom Wappen gilt:

- Wenn die Abkürzung der Zulassungsbehörde ein Zeichen umfasst, dürfen nach dem Wappen minimal 3 bis maximal 6 Zeichen folgen.
- Wenn die Abkürzung der Zulassungsbehörde zwei Zeichen umfasst, dürfen nach dem Wappen minimal 3 bis maximal 5 Zeichen folgen.

Die nach dem Landeswappen folgenden Zeichen bestehen aus einem Ziffernblock gefolgt von einem Buchstabenblock (Großbuchstaben). Die gesamte Anzahl der Zeichen aus Ziffernblock und Buchstabenblock darf den Maximalwert 5 bzw. 6 nicht überschreiten.

Zusatzinformation: Sonderkennzeichen für Behörden bzw. Wunschkennzeichen haben eine andere Struktur und müssen in dieser Aufgabe nicht behandelt werden.

Beispiele für gültige Kennzeichen nach **EU Serie V, einzeilig**:



Aufgaben:

Im File `kfz` befinden sich mehrere KFZ-Kennzeichen. Das Landeswappen wurde der Einfachheit durch ein Blank ersetzt. Suchen Sie alle gültigen KFZ Kennzeichen.

Empfohlene Vorgehensweise :

- a. Formuliere eine `grep`-Anweisung, die prüft, ob die Formatierung vor dem Landeswappen richtig und die gesamte Länge des Kennzeichens im zulässigen Bereich ist. Speichern Sie die Liste der Matches in einer Datei namens `./tmp`.
- b. Formuliere eine weitere `grep`-Anweisung, die als Input die unter a) erzeugte Datei `./tmp` verwendet und die überprüft, ob nach dem Wappen die Reihenfolge Ziffernblock gefolgt von Buchstabenblock (Großbuchstaben) richtig ist. Sowohl der Ziffernblock als auch Buchstabenblock (Großbuchstaben) müssen vorhanden sein.

- c. Versuchen Sie die unter a) und b) erzeugten grep-Anweisungen mit Hilfe einer Pipe zu kombinieren, um die Verwendung des Hilfsfiles ./tmp zu vermeiden.

File k.fz :

PE1246A

L 527DK

LL 1BC

W 12345A

I 1WE3RT

FR 1234ACS

L 3A

PE 32X

L 32UM

Beispiel 1: *mmv*

Erstellen Sie ein Shellsript `mmv.sh` (=multiple move), welches die Extensions von Dateinamen, die sich in im aktuellen Arbeitsverzeichnis befinden, ändert. Gestalten Sie ihr Script benutzerfreundlich, sodass bei inkorrekten Aufrufen (z.B. zu wenige Argumente) mit entsprechenden Fehlermeldungen reagiert.

Beispiel:

```
./mmv.sh htm html # Umbenennen aller Files mit der Extension .htm in .html
```

Anmerkung: Unterverzeichnisse sind nicht zu ändern!

Beispiel 2: *sizes*

Erstellen Sie ein Shellsript `sizes.sh`, das für gegebene Verzeichnisse die Größen aller der darin enthaltenen Files berechnet. Außerdem soll am Ende des Shellscripts die Summe aller berechneten Werte ausgegeben werden. Ein Abstieg in Unterverzeichnisse muss nicht erfolgen.

Folgendes ist zu tun:

- Der Pfad der zu analysierenden Verzeichnisse soll als Argument an das Shellsript übergeben werden.
- Die Argumente sind auf Korrektheit zu prüfen.
- Die Größe von Files kann mit dem `ls`-Kommando ermittelt werden.
- Am Ende sollen die Größen wie folgt ausgegeben werden:

```
Size of files in directory /firstDir: 1024 Bytes
```

```
Size of files in directory /secondDir: 2048 Bytes
```

```
Total Size of Directories: 3072 Bytes
```

- Zum Testen können sie sich mit untenstehenden Befehl Dateien beliebiger Größe – in untenstehenden Befehl z.B. 5 KByte (= 5120 Byte) – erzeugen:

```
head -c 5K < /dev/urandom > myfile
```

Beispiel 3: *curriculum*

Folgendes File curriculum.txt ist gegeben:

```

School of Informatics, Communications and Media
Upper Austria University of Applied Sciences
MBI (BSc)
Lectures Hours per week in semester
Computer Science
Computer Science Introduction;4;0;0;0;0;0
Algorithms and Datastructures;8;8;0;0;0;0
Software Development;0;0;5;5;5;0
Data Engineering;0;4;4;3;0;0
Operating Systems and Network Technology;0;3;0;3;0;0
Web-Techniques;0;0;2;0;0;0
Life Sciences and Math/Statistics
Biolab;0;0;0;2;0;0
Mathematics;4;4;0;0;0;0
Statistics;0;0;4;4;4;0
Chemistry/Biochemistry;3;0;3;0;0;0
Biology;3;0;0;0;0;0
Physiology;0;4;0;0;0;0
Cell Biology;0;0;3;0;0;0
Genetics;0;0;0;2;0;0
Proteomics;0;0;0;0;2;0
Intergrative Lectures
Projectmanagement;2;2;0;0;0;0
Interdisciplinary Project;0;0;3;3;6;0
Internship and Bachelorthesis Seminar;0;0;0;0;0;4
Domainspecific Softwaretools;0;0;3;0;0;0
Domainspecific Algorithms;0;0;0;0;3;0
Signal and Imageprocessing;0;0;0;3;0;0
Machine Learning;0;0;0;0;4;0
Business and Social Skills
Business Administration;0;0;0;2;1;0
Bioethics;0;0;0;0;0;1
English;2;2;1;1;1;0
Law;0;0;0;0;0;1
Social Skills;2;1;0;0;2;0

```

Dieses File zeigt das Curriculum eines Bachelorstudiengangs in Hagenberg. Sie finden eine Liste aller Fächer über das gesamte Studium. Bei jedem Fach steht die Anzahl der Wochenstunden - durch Strichpunkt (;) getrennt - in jedem Semester dabei.

Beispiel: Computer Science Introduction;4;0;0;0;0;0

Das Fach Computer Science hat 4 Stunden im 1.Semester, vom 2.-6. Semester werden für dieses Fach keine Stunden mehr vorgesehen.

Erstellen Sie ein Shell Script `curriculum.sh`, das die Anzahl der Wochenstunden pro Semester berechnet und an den File `curriculum.txt` in folgender Form anhängt.

```
1.Semester: XX Wochenstunden
2.Semester: XX Wochenstunden
3.Semester: XX Wochenstunden
4.Semester: XX Wochenstunden
5.Semester: XX Wochenstunden
6.Semester: XX Wochenstunden
```

Überprüfen Sie, ob die Datei `curriculum.txt` existiert. Er ist nicht als Argument an das Script zu übergeben.

Achtung: Nach dem Einlesen der Variablen von einem File ist zu beachten, dass Variablen, für die beim Lesen keine Werte gefunden wurden, leer sind und ggf. bei Rechenoperationen keinen gültigen Integer Wert enthalten.

Shellprogrammierung

Aufgabe:

a)

Folgender File *studenten* ist gegeben.

```
1410307047 Aberl David
1410307048 Aigner Martin Engelbert
1410307104 Amadi Christoph Chukwuka
1410307050 Baumgartner David
1420307106 Berei Tobias
1410307051 Bernard Michael Pascal
1410307052 Eder Fabian Helmut
1420307107 Enzesberger Hannes
1410307053 Exler Markus Peter Dietmar
1410307055 Gritzer Daniela
1410307056 Golledge Mathew Kyle
1410307105 Haasdyk Timothy Norman
```

Erstellen Sie ein Shell-Script *email*, das eine Datei *stud_email* mit der Email-Adresse der Studierenden nach unten dargestelltem Schema erzeugt!

```
1410307047 S1410307047@students.fh-hagenberg.at Aberl David
1410307048 S1410307048@students.fh-hagenberg.at Aigner Martin Engelbert
1410307104 S1410307104@students.fh-hagenberg.at Amadi Christoph Chukwuka
1410307050 S1410307050@students.fh-hagenberg.at Baumgartner David
1420307106 S1420307106@students.fh-hagenberg.at Berei Tobias
1410307051 S1410307051@students.fh-hagenberg.at Bernard Michael Pascal
1410307052 S1410307052@students.fh-hagenberg.at Eder Fabian Helmut
1420307107 S1420307107@students.fh-hagenberg.at Enzesberger Hannes
1410307053 S1410307053@students.fh-hagenberg.at Exler Markus Peter Dietmar
1410307055 S1410307055@students.fh-hagenberg.at Gritzer Daniela
1410307056 S1410307056@students.fh-hagenberg.at Golledge Mathew Kyle
1410307105 S1410307105@students.fh-hagenberg.at Haasdyk Timothy Norman
```

b)

Erweitern Sie dieses Shellsript *email* so, dass die Mail-Adressen der Studierenden nicht nur auf Basis der Matrikelnummern erstellt werden, sondern auch nach folgendem Schema: **Vorname.Name@students.fh-hagenberg.at**

```
1410307047 S1410307047@students.fh-hagenberg.at David.Aberl@students.fh-hagenberg.at Aberl David
1410307048 S1410307048@students.fh-hagenberg.at Martin.Aigner@students.fh-hagenberg.at Aigner Martin Engelbert
1410307104 S1410307104@students.fh-hagenberg.at Christoph.Amadi@students.fh-hagenberg.at Amadi Christoph Chukwuka
1410307050 S1410307050@students.fh-hagenberg.at David.Baumgartner@students.fh-hagenberg.at Baumgartner David
1420307106 S1420307106@students.fh-hagenberg.at Tobias.Berei@students.fh-hagenberg.at Berei Tobias
1410307051 S1410307051@students.fh-hagenberg.at Michael.Bernard@students.fh-hagenberg.at Bernard Michael Pascal
1410307052 S1410307052@students.fh-hagenberg.at Fabian.Eder@students.fh-hagenberg.at Eder Fabian Helmut
1420307107 S1420307107@students.fh-hagenberg.at Hannes.Enzesberger@students.fh-hagenberg.at Enzesberger Hannes
1410307053 S1410307053@students.fh-hagenberg.at Markus.Exler@students.fh-hagenberg.at Exler Markus Peter Dietmar
1410307055 S1410307055@students.fh-hagenberg.at Daniela.Gritzer@students.fh-hagenberg.at Gritzer Daniela
1410307056 S1410307056@students.fh-hagenberg.at Mathew.Golledge@students.fh-hagenberg.at Golledge Mathew Kyle
1410307105 S1410307105@students.fh-hagenberg.at Timothy.Haasdyk@students.fh-hagenberg.at Haasdyk Timothy Norman
```

c)

Übergeben Sie den File *studenten* als Argument an das ShellScript und überprüfen Sie, ob dieser File auch existiert. Überprüfen Sie auch, ob dem Shellscript überhaupt ein Filename übergeben wurde. Beenden Sie das Shell Script mit einer Fehlermeldung, wenn es diesen File nicht gibt bzw. kein Argument übergeben wurde.

Shellprogrammierung Übungsbeispiele:

Beispiel 1:

Einem Shellscript `file_size` wird ein Verzeichnisname übergeben.

Es ist eine Datei `fsize` im gegebenen Verzeichnis zu erzeugen, die

- a) die Namen aller regulären Dateien in diesem Verzeichnis und die dazugehörige Filegröße enthält ,
- b) keine Unterverzeichnisse im gegebenen Verzeichnis berücksichtigt und
- c) in der letzten Zeile auch die Summe der Größen aller Files enthält, die gefunden wurden.

Am Beginn des Shellscripts ist zu überprüfen, ob die Anzahl der übergebenen Argumente passt und das übergegebene Verzeichnis auch existiert. Weiters ist die Datei `fsize` zu löschen, falls sie schon vorhanden ist.

Beispiel 2:

Im File `wetter` sind außergewöhnliche Wetterereignisse des letzten Monats gespeichert.

Der File `wetter` hat befindet sich in Ihrem Homeverzeichnis und hat folgende Form:

```
04.05.2017 - 16:22 Uhr
Hagel in Brunnhof (Amstetten)
04.05.2017 - 16:20 Uhr
Hagel in Miesenbach bei Birkfeld (Weiz)
23.04.2017 - 12:52 Uhr
Hagel in Aschach an der Steyr (Steyr-Land)
20.04.2017 - 10:12 Uhr
Schnee in Hietzing (Wien)
20.04.2017 - 03:40 Uhr
Schnee in Thenneberg (Baden)
19.04.2017 - 22:15 Uhr
Schnee in Mariazell (Bruck-Mürzzuschlag)
07.04.2017 - 05:59 Uhr
Erdbeben in Semmering (Neunkirchen)
```

Jedes Wetterereignis umfasst zwei Zeilen:

- 1) In der ersten Zeile befinden sich Datum und Uhrzeit des Wetterereignisses.
- 2) In der zweiten Zeile befinden sich das Wetterereignis, der Ort und der Bezirk in runden Klammern. Wetterereignis und Ort sind durch das Wort **in** getrennt. Zwischen dem Ortsnamen und dem Bezirk in runden Klammern befindet sich ein Leerzeichen. Der Ort kann aus mehreren Worten bestehen: z.B.: `Aschach an der Steyr`.
- 3) Es wird nur ein einzelnes Leerzeichen als Trennzeichen zwischen den Feldern verwendet.

Erstellen Sie ein Shellscript `meldungen` mit folgender Funktionalität:

- a) Dem Shellscript soll das Wetterereignis als Argument übergeben werden.
- b) Wenn das übergebene Wetterereignis im File `wetter` gefunden wird, soll ein File `resultat` erzeugt werden, dessen Aufbau im folgenden Beispiel erklärt wird.

Shellscript Aufruf:

`meldungen Hagel`

Ausgabe des Files `resultat` im Shellsript:

Hagel gab es in folgenden Orten:

Brunnhof im Bezirk Amstetten

Miesenbach bei Birkfeld im Bezirk Weiz

Aschach an der Steyr im Bezirk Steyr-Land

Die erste Zeile ist immer die Überschrift:

Hagel gab es in folgenden Orten:

Die weiteren Zeilen umfassen Orte und Bezirke, in den das Wetterereignis stattgefunden hat:

Brunnhof im Bezirk Amstetten

Miesenbach bei Birkfeld im Bezirk Weiz

Aschach an der Steyr im Bezirk Steyr-Land

Dabei ist zu beachten, dass jede Zeile mit dem Namen des Ortes beginnt, in dem das Wetterereignis stattgefunden hat. Dann folgen die Worte **im Bezirk** und dann folgt der Name des Bezirks ohne Klammerung.

- c) Zusätzlich ist zu überprüfen, ob die Anzahl der übergebenen Argumente stimmt und ob der File `wetter` überhaupt existiert. Falls das übergebene Wetterereignis nicht gefunden wird, ist eine entsprechende Fehlermeldung auszugeben. Achten Sie auch darauf, dass das Wetterereignis im File `wetter` immer das erste Wort in der Zeile ist!

